

Data Engineer — Take-Home Assignment

Britannica Home Fashions (BHF) · Tech Team

Objective

BHF receives vendor purchase orders as semi-structured text and needs them turned into clean, structured spreadsheets — and eventually loaded into our systems — for downstream use. This exercise is a small, realistic version of that pipeline: parse a messy source document, produce validated structured output, and take a first pass at loading it reliably.

This is scoped for a Data Engineer role, so beyond parsing we're also looking at how you handle real-world messiness, repeatability, and observability — the things that matter once a script like this needs to run unattended, night after night.

What you're given

- `purchase_order_sample.txt` — a sample vendor purchase order
- `purchase_order_sample_2.txt` — a second vendor PO with a different layout/field order
- `output_template_blank.xlsx` — the exact column headers your output must match

Core Task

1. Write a script that reads a purchase order text file and extracts the header fields (buyer, ship terms, PO#, ref master PO#, ship date, vendor, ship to, bill to) and every line item (department, SKU, UPC, vendor part number, description, retail, cost, ext cost, cartons, case pack, ext quantity, cube, kilograms).
2. Output one row per SKU into a spreadsheet matching `output_template_blank.xlsx` exactly, including header fields repeated on every row.
3. Validate the numbers: recompute EXT QTY ($\text{cartons} \times \text{case pack}$) and EXT COST ($\text{cost} \times \text{ext qty}$), and flag any row where your recomputed value doesn't match what's printed on the PO.

Added for this round

1. Handle messiness explicitly, don't just skip it

`purchase_order_sample.txt` has three deliberately broken things planted in it:

- SKU 71097218 is missing its UPC continuation line entirely.
- SKU 71293252 has a printed EXT COST (4146.080) that doesn't match $\text{cartons} \times \text{case pack} \times \text{cost}$ (which computes to 4046.080).
- SKU 71293276 has a malformed COST field — printed as "55,440" instead of "55.440".
- For each, decide and document: skip the row, flag it, best-guess it, or fail the whole file — and say why. Don't silently drop bad rows; we want to see the decision, not just clean output.

2. Handle a second vendor layout

`purchase_order_sample_2.txt` is a different vendor with a different layout: header fields are labeled and ordered differently, Bill To/Ship To are swapped, and the UPC is printed inline per item rather than on its own line below. Your script should handle both files without being a hardcoded, single-shot parser for sample 1 only.

- In your README, note what your parser assumes is fixed vs. flexible, and what would break if a third vendor sent yet another format tomorrow.

3. Add a loading step

Beyond the spreadsheet, write a script (or SQL DDL + load script) that loads your output into a SQL table (SQLite is fine — no need to stand up SQL Server).

- Design it so running the load twice on the same file does not create duplicate rows — show us it's idempotent (upsert on a natural key, truncate-and-reload, or similar).
- Briefly explain your dedupe/upsert strategy in the README.

4. Add a simple log / audit trail

Output a log (console or file) noting: rows processed, rows flagged for mismatch, and any parsing errors encountered.

- Assume this runs unattended overnight — how would you know something went wrong without opening the output file yourself?

README — please include

- Your approach and any assumptions made
- How you handled each messy/edge case (see #1)
- Your dedupe/idempotency strategy for the load step (see #3)
- What you'd change or add with more time
- **If you used an AI coding assistant:** include whatever instruction/context file it uses (CLAUDE.md, AGENTS.md, .cursorrules, or equivalent), committed as-is. If your tool doesn't use a file like that, describe roughly how you directed it.
- **One thing the AI got wrong:** if you used an assistant, note one thing it produced that you had to correct or override. If genuinely nothing needed correcting, say so directly.

Deliverables

- Script(s) in a public GitHub repository
- The generated output spreadsheet(s), committed to the repo
- The load script / SQL, committed to the repo
- A README covering the points above

Evaluation criteria

- Correct field extraction, including the UPC line that sits under each item row
- Output matches the required schema and values
- Sensible, explicit handling of the validation and edge-case steps — not just clean-path code
- Parser isn't hardcoded to a single vendor layout
- Idempotent, sensible load design
- Code clarity and organization
- README quality and honesty, including AI-usage notes

Notes

- Use whatever tools you like, including AI assistants — we're as interested in how you direct and check the work as in the output itself.
- Aim for 3–4 focused hours. Don't over-engineer — we want to see how you approach one real, slightly messy problem, not a production system.
- Once submitted, we'll do a short live walkthrough where we ask you to explain your own code and adapt it to a new requirement on the spot.